Assignment 2 Report: Deep Learning Fundamentals

Student ID: 1936708
Master of Data Science, University of Adelaide

01/11/2025

1 Introduction

Assignment 2 aims to build a convolutional neural network, which is used to predict the probability of an individual get diabetes when meeting several clinical criteria. The datasets used for training and testing encompass eight input variables and a target variable named `Outcome`. Assignment 2 is binary classification cause the label 1 indicates a positive case and 0 indicates a negative case. The target of assignment 2 is to preprocess the *traindata.csv* and *testdata.csv*, using PyTorch to design a neural network model, and using multiple parameter settings evaluate its performance.

2 Method

2.1 Data Preprocessing

Both of the *traindata.csv* and *testdata.csv* include eight numerical attributes such as `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI`. Certain columns contain zero values, which were interpreted as missing entries and replaced with the median of the non-zero values in that column. Because `Glucose` and `BloodPressure` can't be 0(that means people was dead), `SkinThickness` and `Insulin` also can't be 0(that means not measure) and `BMI` should greater then 10 in usual. All features were then standardized using z-score normalization:

$$z = \frac{x - \mu}{\sigma}$$

to make each feature comparable and to stabilize model training. The cleaned data were randomly divided into training and validation sets in an 80%–20% ratio.

2.2 Model Implementation

A simple fully connected neural network (Multi-Layer Perceptron) was implemented from scratch using the `torch.nn.Module` class. The model consists of one or more hidden layers

with the ReLU activation function. The output layer predicts two classes (0 or 1). The loss function used was cross-entropy, and the optimizer was Adam. During training, the gradient of previous batches was cleared each time by calling `optimizer.zero_grad()` to avoid gradient accumulation errors.

3 Experiments and Results

Three sets of hyperparameters were tested, varying the number of hidden layers and learning rates. Each configuration was trained for 30–50 epochs, and validation accuracy was used to select the best-performing model.

Set	Hidden Layers	Learning Rate	Validation Accuracy
1	(32)	0.001	0.75
2	(64, 32)	0.0005	0.54
3	(128, 64, 32)	0.001	0.63

Table 1: Comparison of different hyperparameter configurations.

The best configuration (Set 1) achieved 0.75 validation accuracy and 0.93 test accuracy. The model weights were saved using the following command:

```
torch.save(best_model.state_dict(), "best_mlp_diabetes.pth")
```

which allows the trained model to be reloaded and tested again without retraining.

4 Discussion and Conclusion

The results show that the Adam optimizer provides faster and more stable convergence compared with other optimizers tested. Gradient cleaning before each parameter update is critical to prevent incorrect weight updates due to accumulated gradients. Overall, the implemented neural network achieved satisfactory performance within the assignment's scope. Possible future improvements include adding regularization or experimenting with different network depths and learning rates.